人工智能应用实践系统实现报告

面向招聘场景的智能人才评估与推荐平台

ScoutAgent：基于 LangGraph 的多智能体协作系统

组　员： 郑吴穷　于笑　李汉尧

学　号： 231380128　231380124　220820114

学　院： 信息与智能科学学院

专　业： 智能科学与技术

课程名称： 智能技术工程实训

报告类型： 系统实现报告

日　期： 2026/4/20

东华大学

信息与智能科学学院

摘　要

当前互联网招聘场景普遍存在四类难以回避的痛点：基于关键词的简历筛选无法刻画真实技能匹配、候选人对简历的过度包装难以察觉、技术能力评估缺乏可量化的过程指标、多轮评估信息散落且难以沉淀为组织资产。本文以 **ScoutAgent**——一个面向技术招聘场景的多智能体协作评估平台为对象，给出从行业痛点到工程落地的完整方案。系统以 **LangGraph** 为编排核心，对外暴露 **ReAct** 与 **Plan-and-Execute** 两套图，统一组合 **Tools / Skills / MCP** 三层工具体系；在记忆侧采用 **Redis Checkpointer** 维护工作记忆、**PostgreSQL** 持久化企业级与岗位级偏好；在算法侧落地了三项关键技术：基于通义千问与思维链的结构化抽取（F1 由 66% 提升至 78%）、基于 **Neo4j** 的技术知识图谱与 **GraphRAG** 匹配增强（JD-CV 二分类准确率由 74.3% 提升至 85.2%），以及面向简历过度包装检测的 **SFT+DPO** 两阶段对齐（Pairwise Accuracy 由 73.5% 提升至 88.5%）。系统采用 React + FastAPI + 三类存储 + Docker Compose 的工程拓扑，已在阿里云 ECS 与 AutoDL GPU 之间通过 SSH 端口转发完成端到端联调。本报告分五章依次呈现选题背景与需求、Agent 架构设计、关键算法与实验验证、技术栈与部署、以及前端系统功能展示。

关键词：LLM Agent；LangGraph；GraphRAG；知识图谱；SFT；DPO；招聘评估

目录

1 选题背景与需求分析 4
1.1 招聘场景的核心矛盾 4
1.2 招聘流程中的四类核心痛点 4
1.3 从行业痛点到技术选型的反向推演 5
1.4 为什么必须是 Agent，而不是单次 LLM 调用 6
1.5 选题立意与价值定位 6

2 Agent 架构设计 7
2.1 整体架构总览 7
2.2 Function Calling：三层工具体系 8
2.3 Memory 体系：短时工作记忆 + 长期偏好沉淀 11
2.4 Planning：ReAct 与 Plan-and-Execute 双范式 12

3 关键算法与实验验证 14
3.1 算法体系概览 14
3.2 结构化信息抽取（Qwen API + CoT） 15
3.3 技术知识图谱与 GraphRAG 匹配增强 16
3.4 简历过度包装检测（SFT + DPO） 18
3.5 微调模型服务化部署 19

4 技术栈与部署 20
4.1 总体架构 20
4.2 前端 20
4.3 后端 API 21
4.4 Agent 框架 22
4.5 数据存储 23
4.6 部署 24
4.7 云端运行环境与 AutoDL 推理隧道 24

5 系统功能展示 25
5.1 页面总览 25
5.2 企业与岗位管理 27
5.3 候选人管理 28
5.4 批量化评估看板 29
5.5 深度评估 Agent（重点） 30
5.6 Eval 系统 33

6 结论与展望 **33**
6.1 核心结论 34
6.2 局限与未来工作 34

1 选题背景与需求分析

1.1 招聘场景的核心矛盾

招聘是一个**信息密度高、判断成本重、容错率低**的典型业务场景：岗位描述（JD）通常以结构化的硬性要求与软性偏好呈现，而候选人简历（CV）却是高度个性化的非结构化叙述。HR 每天的工作，本质上就是在大量异构文本之间寻找匹配、辨别真伪、给出综合判断——这正是大语言模型最适合发挥的场景。然而，当前主流的 ATS（Applicant Tracking System）仍以关键词过滤为主，距离”理解候选人”还有相当差距。

ScoutAgent 的选题正是从这一矛盾出发：把 HR 工作中”最重复、最易错、最难沉淀”的环节交给 Agent，让人回到真正需要判断的位置。

1.2 招聘流程中的四类核心痛点

我们将企业实际招聘流程拆解为初筛、深筛、技评、终评四个阶段，分析每个阶段 HR 实际承担的工作以及对应的痛点（见表 1）。

表 1: 招聘流程各阶段的 HR 实际工作与主要痛点

阶段	HR 实际承担的工作	主要痛点
初筛	关键词搜索简历池	同义词、上下位关系、技术栈迁移识别能力差
深筛	阅读项目经历与经历叙述	包装、虚标、年限注水难以辨别
技评	出题、阅卷、看代码	题目同质化、判分主观、无法考察过程
终评	多轮拍板、形成结论	信息散落多处，难以复盘与沉淀经验

四个阶段的痛点不是孤立存在，而是从前到后串成一条流水线（见图 1）。每一处痛点都会在下一阶段被放大：初筛把人选错了，深筛就要花更多时间补救；深筛没识破包装，技评和终评就只能在错误的样本池里挑选。

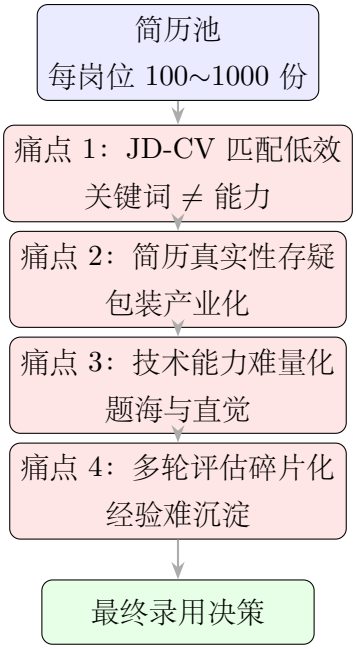

图 1: 招聘流程中的四类核心痛点流水线

痛点一：关键词匹配 ≠ 能力匹配。 纯文本匹配天然处理不了”语义 + 结构”的双重对齐：技术栈存在大量**同义词**（K8s 与 Kubernetes）、**上下位关系**（Spring Cloud 属于微服务）和**迁移能力**（具备 Java 后端经验大概率能上手 Go 后端）。

痛点二：简历”通用化包装”已成产业。 市面上充斥着”简历优化教程”，候选人往往针对 JD 反向定制简历，常见手法包括职责膨胀、技术堆砌、时间含糊与项目挪用。单点信息无法证伪，只能通过**跨字段交叉验证**才能识别。

痛点三：技术能力评估靠”题海 + 直觉”。 传统笔试与白板题与真实工程能力相去甚远，HR 真正想知道的是候选人的解题路径、代码结构和被追问后的反应，但现有平台只能给出”通过率”，无法提供过程评估。

痛点四：多轮评估信息”碎片化”。 初筛 Excel、面试官口头反馈、技术评分表、HR 邮件等信息散落在多个系统中，最终决策时拼不起来；HR 一旦离职，组织对该岗位的偏好积累也随之消失。

1.3 从行业痛点到技术选型的反向推演

针对上述四类痛点，本系统在技术选型上做了一一对应的设计（见表 2）。整体思路是：与其面向”AI 简历筛选器”做单点替代，不如面向”AI HR 助手”做**增强式协作**——把 Agent 嵌入 HR 现有的工作流，让其调用知识图谱与外部工具、记住每个客户的偏好、按计划完成多步评估，并把全过程以可追溯报告的形式沉淀下来。

表 2: 行业痛点与技术选型的对应关系

行业痛点	技术选型	核心思路
同义词 / 上下位概念	Neo4j 知识图谱 + GraphRAG	把" 技术栈" 建成图，用结构推理代替字符串匹配
包装与虚标	真实性检测 Agent	LLM 跨"JD / CV / 项目 / 测试" 四源比对
能力评估粗糙	代码执行工具 + 过程评估	Agent 调工具拿运行结果，结合解题过程评分
决策碎片化	Plan-and-Execute + 报告生成	拆解评估流程，输出结构化 Markdown 报告
经验流失	企业级 / 岗位级偏好库	LLM 自动识别硬性要求并 upsert 到 PG

1.4 为什么必须是 Agent，而不是单次 LLM 调用

回到技术路线本身：在 JD 与 CV 文本上做一次 prompt 调用，让 LLM 直接给出匹配分，是否就够用？我们认为不够。招聘任务的特征决定了它必须以 Agent 形态实现：

- 需要**外部能力**：知识图谱、代码沙箱、第三方系统等都不可能塞进 prompt，必须通过 **Function Calling** 在运行时调用；
- 需要**跨会话记忆**：同一家客户的招聘偏好、同一个岗位的硬性要求需要被记住，必须有 **Memory** 层；
- 需要**多步骤拆解**：先匹配再验真再技评再综合，需要 **Planning** 控制执行流；
- 需要**可解释性**：HR 必须能看到 Agent 调了哪些工具、走过哪些步骤，才能信任最终结论。

单次问答只能解决" 问题"，Agent 才能解决" 流程"。这正是 ScoutAgent 必须以 Agent 架构落地的根本原因，也奠定了第 2 章的设计基调。

1.5 选题立意与价值定位

在产品定位上，ScoutAgent 与传统简历筛选工具有本质差异（见表 3）：传统工具的目标是" 提升处理速度"，本系统的目标是" 提升判断质量"；传统工具输出一个分数，本系统输出一份带证据链的报告；传统工具替代 HR，本系统增强 HR。

表 3: ScoutAgent 与传统筛选工具的定位对比

维度	传统筛选工具	ScoutAgent
目标	提升处理速度	提升判断质量
输出	一个匹配分数	一份带证据链的结构化报告
对 HR 的关系	替代 / 引发被取代焦虑	协作式增强
可解释性	黑盒打分	计划 + 工具调用全过程可见
经验沉淀	无	企业级与岗位级偏好持续学习

进一步从价值闭环看，本项目同时具备技术、业务与社会三层意义：技术层面在真实业务中验证 Function Calling、Memory、Planning 三大模块的协同；业务层面把 HR 每岗位评估时间从”几小时/份”压缩到”几分钟/份”并产出可追溯报告；社会层面降低”靠包装上岗”与”靠学历过滤”的概率，让真实能力被看见，让非典型背景候选人获得更公平的机会。

2 Agent 架构设计

2.1 整体架构总览

ScoutAgent 后端 Agent 基于 **LangGraph** 构建，对外暴露两套并列的图：用于实时招聘对话的 **Chat Graph**（ReAct 范式）与用于复杂任务编排的 **Planning Graph**（Plan-and-Execute 范式）。两套图通过统一的工具解析层 `tool_resolver` 在运行时动态绑定能力，并共用同一套两层记忆体系（短时工作记忆 + 长期偏好沉淀）。整体结构如图 2 所示。

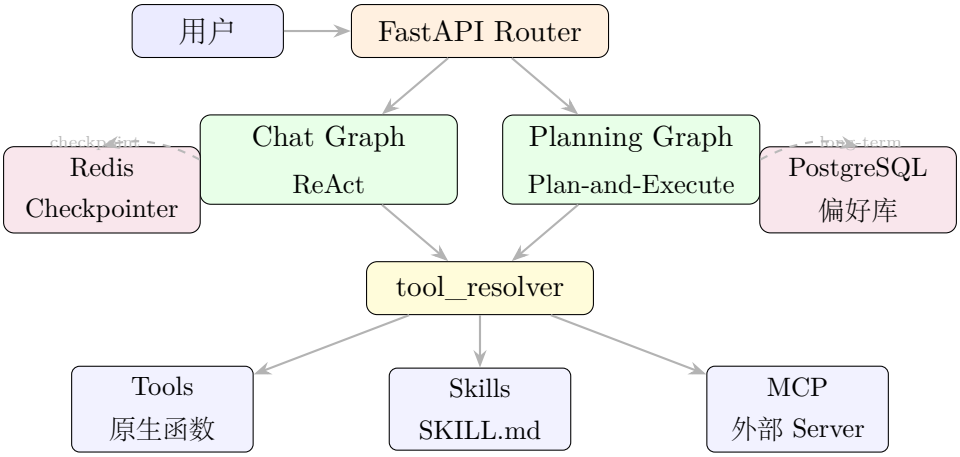

图 2: ScoutAgent 后端 Agent 整体架构总览

2.2 Function Calling：三层工具体系

把”可被 LLM 调用的能力”做了清晰的分层是 ScoutAgent 在 Function Calling 设计上的核心抽象：每一层职责不同，但通过统一的 `resolve_tools(config)` 在每次推理前组装为一份扁平的工具列表，再交给 LLM 绑定。三层结构如图 3 所示，三层之间的对比见表 4。

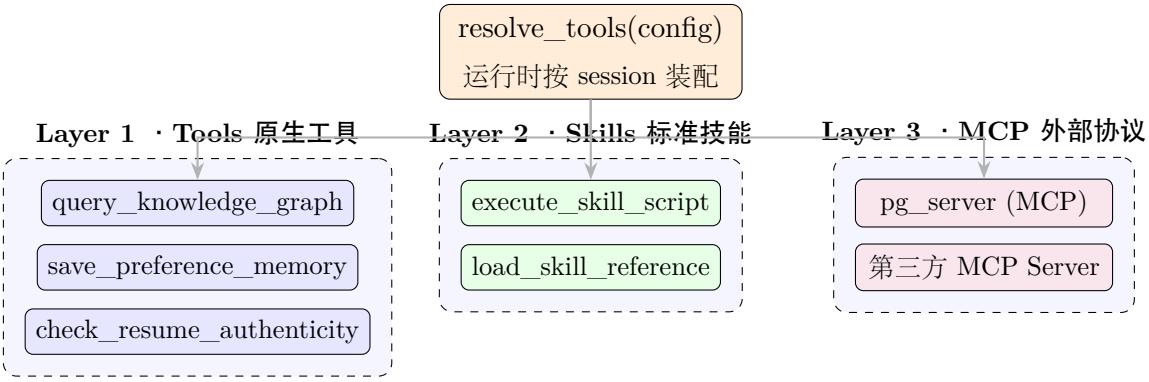

图 3: Function Calling 三层工具体系

表 4: Tools / Skills / MCP 三类工具对比

维度	Tools	Skills	MCP
定位	业务原子能力	领域专家手册 + 可选脚本	外部进程能力
载体	Python 函数（StructuredTool）	SKILL.md 目录	独立进程（stdio / http / sse）
注册方式	模块导入 BASE_TOOLS	YAML frontmatter 扫描	mcp_servers.json
发现时机	启动时静态导入	启动时扫描目录	启动时连接 + get_tools()
典型例子	KG 查询、保存偏好、真实性检测	jd-cv-match、authenticity-check	Postgres MCP Server

2.2.1 Tools：原生函数工具

最底层的 Tools 直接以 Python 函数 + Pydantic Schema 实现，注册路径在 `backend/app/agent/tool_resolver.py`：

```
BASE_TOOLS = [query_knowledge_graph, save_preference_memory,
check_resume_authenticity]
```

每个 Tool 都被封装成 `StructuredTool`，LLM 看到的只有 `name`、`description` 与 `args_schema`，看不到具体实现，从而保证调用的安全性与可替换性。

2.2.2 Skills：渐进披露的标准技能

Skills 是一种”约定大于配置”的能力封装，每个 Skill 是一个目录，包含 `SKILL.md`（YAML frontmatter + 主体指令）、`references/` 与 `scripts/`。Skills 的关键设计是**渐进披露**（progressive disclosure）：始终把所有 Skill 的元数据放进 system prompt，让 LLM 知道有哪些能力存在；当用户激活后再注入完整 `SKILL.md` 主体；引用文档与脚本则按需通过工具调用加载（图 4）。这样 system prompt 始终保持紧凑，只在确实需要时才”展开”详细内容。

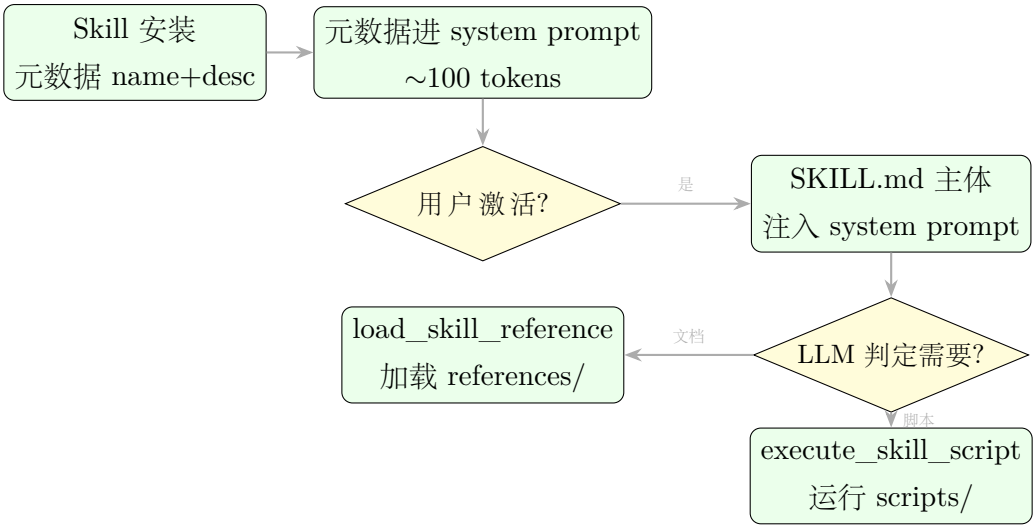

图 4: Skills 渐进披露的三档加载策略

2.2.3 MCP：外部协议层

MCP（Model Context Protocol）把”工具”做成独立进程，用统一协议通信，便于把数据库、第三方系统等异构能力以最小耦合的方式接入 Agent。MCP Server 的注册靠 `mcp_servers.json` 声明式配置；启动顺序与连接流程如图 5 所示。

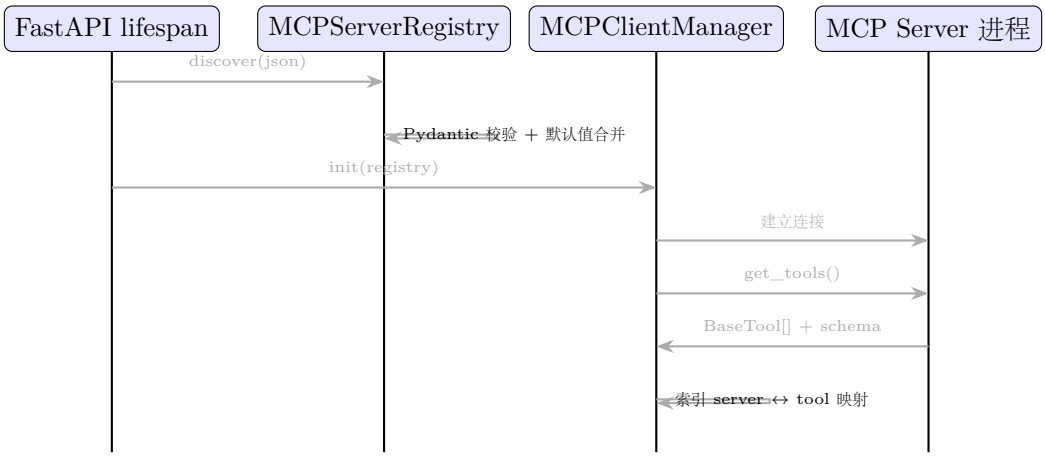

图 5: MCP Server 注册与连接时序

2.2.4 统一解析：运行时按需绑定

无论 Tools、Skills 还是 MCP，最终都要在 LLM 调用前合并为一份扁平的工具列表。`resolve_tools()` 根据当前 `config.configurable` 动态决定：

- 永远包含 `BASE_TOOLS`；
- 若当前 session 激活的 Skills 含脚本/参考文档，则附加 `execute_skill_script` / `load_skill_reference`；

- 根据 `mcp_tags` 过滤 MCP 工具，未指定时则全部接入。

这种”运行时绑定”使得**同一张图、不同 session 可激活不同 Skills 与 MCP 标签**，做到能力的按需组装。

2.3 Memory 体系：短时工作记忆 + 长期偏好沉淀

ScoutAgent 把记忆分成两层（图 6）：**短时工作记忆**解决”这次对话的上下文”，由 LangGraph 的 Redis Checkpointer 自动持久化；**长期偏好沉淀**解决”这家客户、这个岗位的画像”，由 PostgreSQL 持久化并在每次会话启动时注入 system prompt。

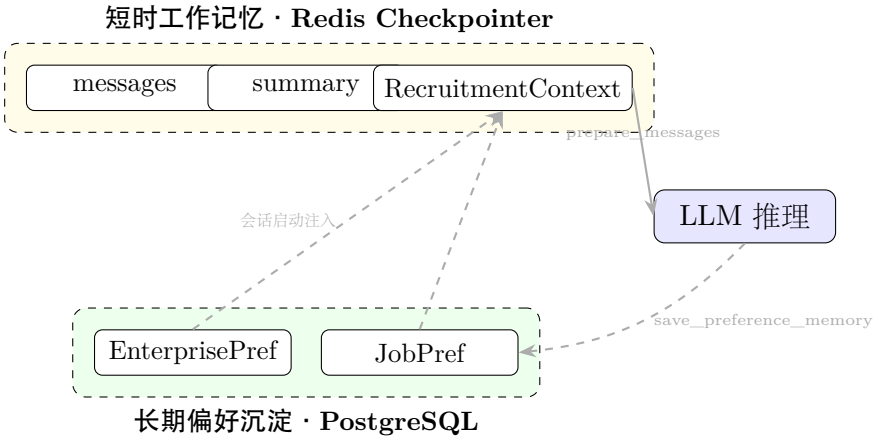

图 6: ScoutAgent 两层记忆体系

2.3.1 工作记忆：状态结构与上下文裁剪

LangGraph 在每一步执行结束后会把整个 state 写入 Redis Checkpointer（带 TTL），下次同 `thread_id` 进入时自动恢复。状态字段在两套图之间共享一个抽象基类 `RecruitmentContext`，包含 `jd_context`、`cv_context`、`assessment_context`、`preference_context` 四个上下文字段，外加 Chat Graph 的 `messages` 与 `summary`、Planning Graph 的 `plan` 与 `past_steps`。

每次 LLM 调用前，`prepare_messages` 会根据 `context_window_budget` 做**优先级裁剪**：(1) system prompt 始终保留；(2) 历史摘要 `summary` 始终保留；(3) 最近 K 轮（`max_recent_turns`）优先放入；(4) 更早的轮次有余量再放；(5) 最后预留 `_REPLY_RESERVE = 2048` tokens 给模型回复。

当 `messages` 数量超过 `summary_trigger_threshold` 时，`maybe_summarize` 节点会把”更早的对话”压缩进 `summary` 字段，原始 message 通过 `RemoveMessage` reducer 从 state 中删除——既保留语义信息，又彻底回收 token。

2.3.2 长期偏好：企业级与岗位级双 scope

长期偏好被建模为两级 scope：**企业级**（跨岗位通用，如”必须 985 学历”）和**岗位级**（特定 JD，如”该岗位必须懂 Go”）。两者都有 `(target_id, preference_key)` 唯一约束，写入用 PostgreSQL 的 `ON CONFLICT DO UPDATE` 实现 upsert，保证同一偏好键覆盖更新（图 7）。

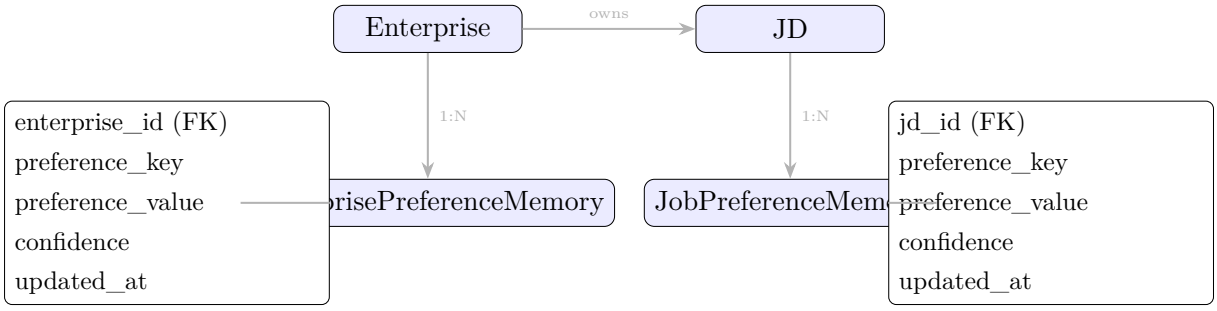

图 7: 企业级 / 岗位级偏好的实体关系

写入路径由 LLM 主动调用 `save_preference_memory` 工具完成。工具描述特意做了行为约束：**只在用户明确表达硬性要求时调用，对试探性表达不调用**——把”什么是值得沉淀的偏好”的判断权交给 LLM 自己。**读取路径**则是在每次会话启动时从数据库重新加载偏好，拼成 markdown 列表写入 `state.preference_context`，再 render 进 system prompt；放在 system prompt 而非 message 历史里，避免被摘要节点误压缩。

2.4 Planning：ReAct 与 Plan-and-Execute 双范式

ScoutAgent 同时实现了两种主流的 Agent 编排范式，分别承担不同的任务负载：**ReAct** 用于实时对话和单点提问，**Plan-and-Execute** 用于多步骤分析和报告生成。两者共享同一套工具与记忆，但执行结构截然不同。

2.4.1 ReAct：经典 Reason–Act–Observe 循环

Chat Graph 是经典的 ReAct 实现，LangGraph 用 `tools_condition` 判断 LLM 输出里是否含 tool call 来决定走向。结构如图 8 所示。每次进入 `chatbot` 节点都重新 `resolve_tools(config)` 并 `bind_tools`，实现”运行时绑定”；`dynamic_tool_node` 同样动态构造 `ToolNode(tools)`，与 chatbot 共用同一份工具列表。失败时通过 `ainvoke_with_fallback` 自动切换备用模型。

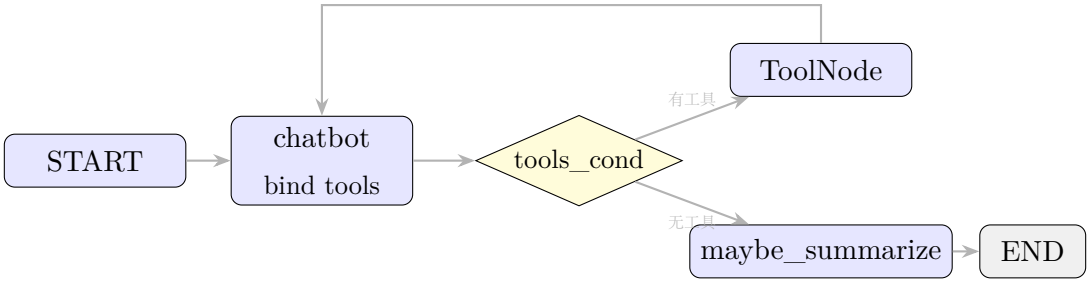

图 8: Chat Graph：ReAct 编排结构

2.4.2 Plan-and-Execute：先规划、再逐步执行、再重规划

Planning Graph 把任务拆成" 先规划 → 逐步执行 → 再规划" 三个阶段，每一步执行内部又是一个小型 ReAct 子图（图 9）。三个核心节点的角色见表 5。

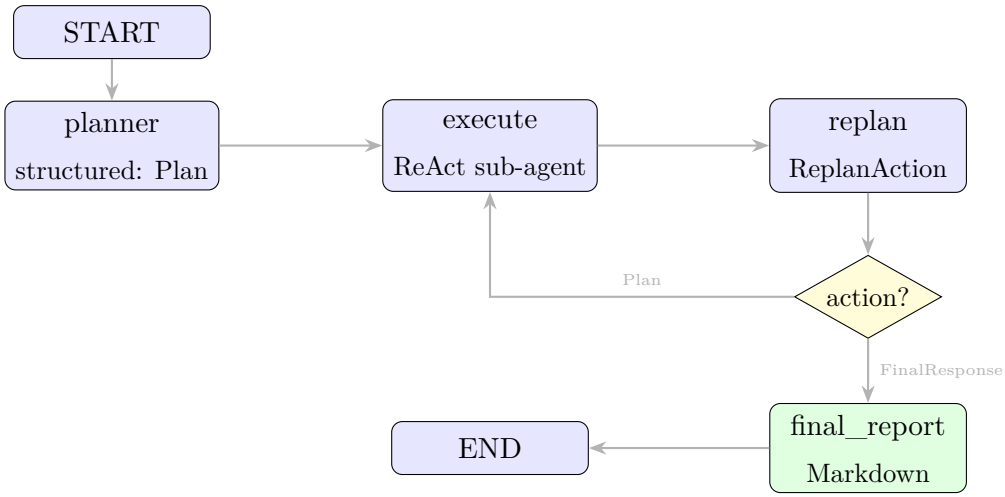

图 9: Planning Graph：Plan-and-Execute 编排结构

表 5: Plan-and-Execute 三个核心节点

节点	输入	输出（Pydantic）	作用
plan_step	user input + 上下文	Plan{steps}	一次性产出有序计划
execute_step	当前 plan 第一项	past_steps += [(step, result)]	跑完一整步 ReAct 子循环
replan_step	已执行 + 剩余计划	ReplanAction (FinalResponse / Plan)	决定继续还是收尾
final_report	全部 past_steps	整合好的 Markdown	输出最终结构化报告

关键设计上有四点值得强调：(1) **结构化输出**——`Plan` / `ReplanAction` / `FinalResponse` 都是 Pydantic 模型，用 `with_structured_output()` 强约束 LLM 输出；(2) **执行器是子图**——`execute_step` 内部用 `create_react_agent` 临时构造 ReAct 子图跑当前步骤，并用 `MAX_EXECUTOR_ITERATIONS=15` 防止递归爆炸；(3) **append-only state**——`past_steps` 用 `Annotated[list, operator.add]`，每个节点只追加自己的结果，避免并发写冲突；(4) **可中断恢复**——与 Chat Graph 共用同一个 Redis Checkpointer，长任务可跨进程恢复。

2.4.3 两种范式的取舍

ReAct 与 Plan-and-Execute 各有适用边界，我们从五个维度做对比（表 6）。最终在 ScoutAgent 中：**深度评估 Agent 默认走 ReAct**，可由用户在前端切换 Plan 模式后改用 Planning Graph。

表 6: ReAct 与 Plan-and-Execute 五维对比

对比项	ReAct	Plan-and-Execute
规划粒度	隐式、每步重新决定	显式、提前一次性产出
上下文消耗	每步把全部历史塞进去	计划字符串短，子执行器隔离
适用任务	短交互、单跳工具调用	多步分析、跨工具协作、生成报告
可解释性	隐藏在 message 流中	plan 数组对外可见
风险	长链路时易绕圈	计划过度刚性，需 replan 弥补

3 关键算法与实验验证

3.1 算法体系概览

围绕" 真实性验证 + 能力评估" 的核心目标，本系统在算法侧落地了三项关键技术（图 10）：(1) 基于通义千问与思维链的**结构化信息抽取**，把非结构化简历与岗位描述转化为标准 JSON 画像；(2) 基于 Neo4j 的**技术知识图谱与 GraphRAG**，用图结构推理代替字符串匹配；(3) 基于 Qwen3.5 4B 的**简历过度包装检测**，采用 SFT 与 DPO 两阶段对齐输出可复核分析。每项技术都设计了与之配套的对照实验，下面分小节展开。

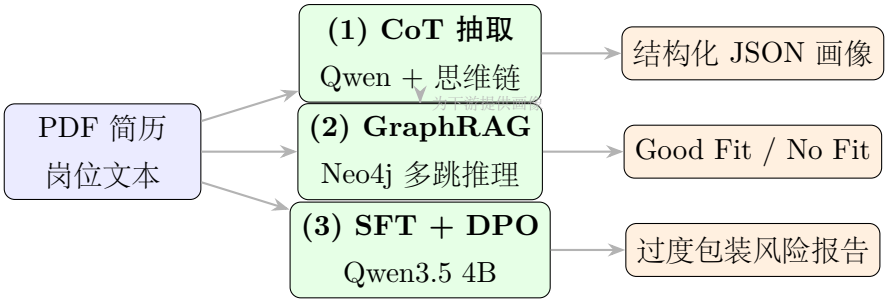

图 10: 三项关键算法及其输入输出关系

3.2 结构化信息抽取（Qwen API + CoT）

3.2.1 方法要点

岗位描述与简历多为非结构化文本，直接匹配噪声大。我们首先用 `pdfplumber` 把简历 PDF 转为纯文本，再调用**通义千问（Qwen）API**，配合**思维链（Chain of Thought）**采用”由粗到细”的两阶段抽取：第一阶段先做**语义分块**（简历六类区块；JD 拆为教育/技能/经历要求等），第二阶段在每个块上按 **Schema** 输出 JSON。简历侧从 NER 14 类标签中选取核心映射：`DESIGNATION` → 期望职位、`EDUCATION` → 教育、`SKILL` → 专业技能、`EXPERIENCE` + `COMPANY` → 实习/工作经历，最终输出统一的候选人画像 JSON。

3.2.2 评测设置与结果

简历评测使用 Kaggle 公开 Resume NER Training Dataset 中的 **5,960** 个样本；岗位评测使用约 **100** 条真实招聘平台非结构化文本与人工提取特征。由于系统输出为 JSON 聚合字段，与序列标注位对齐困难，验证逻辑采用”语义关联比对”——抽取内容与原文标注位置文本相似且信息一致即计为有效召回，各核心维度加权平均，最终汇报 Precision、Recall、F1 三项指标（表 7）。

表 7: 结构化信息抽取性能对比（简历评测，n = 5,960）

方案	Precision (%)	Recall (%)	F1 (%)
Naive LLM 直接抽取（Zero-shot）	68	64	66
Qwen API + CoT 任务分解（本系统）	**79**	**76**	**78**

结论：相对 Zero-shot 直接抽取，CoT 任务分解将 F1 从 66% 提升到 78%，验证了”分块定位 + 局部 Schema 抽取”对长文本”中间信息丢失”问题的缓解效果。

3.3 技术知识图谱与 GraphRAG 匹配增强

3.3.1 图谱构建

技术知识图谱存储于 **Neo4j**，节点以技术 Tag 为主，关系类型与权重经反复调试后固定为表 8 三类。其中跳数衰减满足 $\delta(1) = 1.0, \delta(2) = 0.5, \delta(3) = 0.25$，三跳即截断。

表 8: 知识图谱关系类型与权重

关系名称	关系类别	$w(r)$	业务语义
BELONGS_TO	技术归属 / 分类	5.0	大类—细分，用于多跳扩展
DEPENDS_ON	技术依赖	3.0	强约束依赖链，一跳优先
RELATED_TO	共现关联	1.0	弱关联，权重最低

数据来源覆盖四个渠道：**Stack Overflow Tag Synonyms** 提供异名归一、**GitHub Awesome 系列**（Awesome-Python / Go / Java 等）提供 BELONGS_TO 层次、**Maven 与 PyPI 制品元数据**提供 DEPENDS_ON、**热门仓库 Topics 共现统计**提供 RELATED_TO（带阈值抑制噪声）；额外通过 GitHub API 爬取 Top 约 5,000 仓库标签用于共现与覆盖分析。最终全图规模约为**核心实体 4,800 个、三元组 32,000 条**。

3.3.2 GraphRAG 匹配判别流程

完整的”解析 → 图谱 → 判别”流程如图 11 所示。完整 JD/CV 文本先经 **DeepSeek Chat** 解析为 JSON，再抽取技能列表去重；随后做别名归一并通过 Neo4j **最短路径 ≤3 跳**生成”KG 技能覆盖分析”文本；最后把 JSON 与 KG 文本一并送入 LLM 输出 Good Fit / No Fit。右侧对照分支为”纯 LLM 仅 JSON”，与本路径仅在是否注入 KG 文本上不同，保证对照公平。

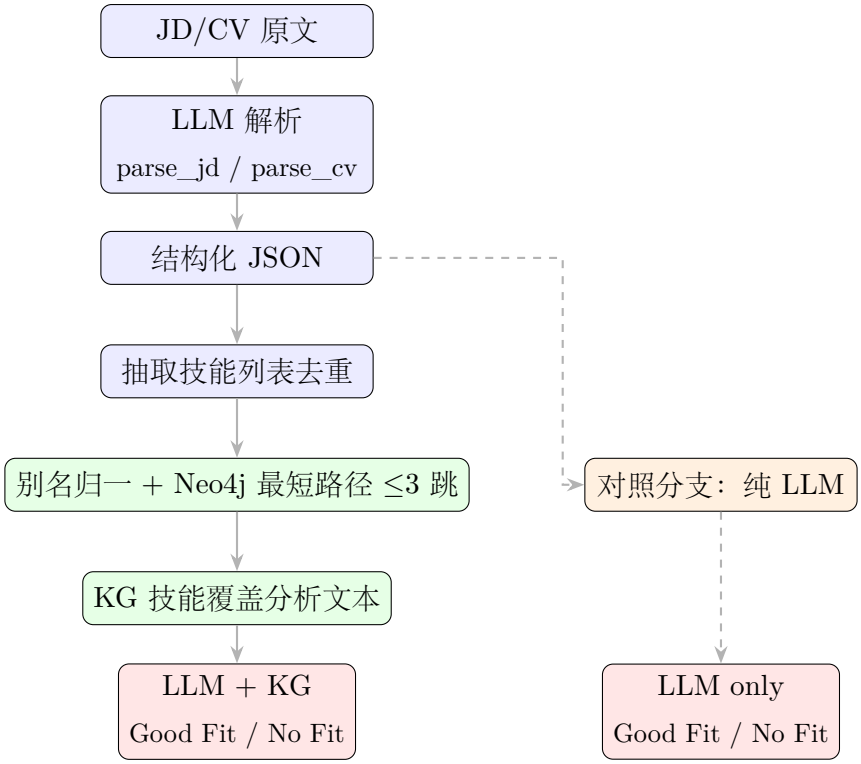

图 11: GraphRAG 匹配判别流程及对照分支

3.3.3 实验一：留一技能补全

第一组实验在 `jd_keywords` 上做 **Leave-One-Out**（LOO）：每次抽掉一个目标技能，用其余技能作为上下文，检验目标技能是否进入推荐 Top-K。基线包括同源 Neo4j 统计的**共现矩阵**与**全局词频**两种朴素方法，指标为 Hit@5 / Hit@10 / MRR（表 9）。

表 9: 留一技能补全的对照结果

方法	**Hit@5** (%)	**Hit@10** (%)	**MRR** (%)
知识图谱多跳推荐（本系统）	**23.32**	34.69	**15.99**
共现矩阵基线（cooccurrence）	22.46	**35.59**	14.07
全局词频基线（freq）	19.61	32.06	11.99

结论：图谱多跳推荐在 Hit@5 与 MRR 上均优于共现与词频基线，证明了图结构在”语义近邻”识别上的优势；Hit@10 上略低于共现矩阵，原因是共现统计在长尾时召回更广，但这部分召回的相关性更弱（MRR 较低也佐证这一点）。

3.3.4 实验二：JD/CV 匹配二分类

第二组实验直接在匹配判别任务上评测。数据来自 Hugging Face 公开数据集 `facehuggerapoorv/resume-jd-match`（约 8k 行），清洗后约六千余条有效记录，字段包含 `cv`、`jd`、`label`（No Fit / Good Fit / Potential Fit）、`cv_keywords`、`jd_keywords`。评测时**排除 Potential Fit**，仅在 Good Fit 与 No Fit 上做二分类，对比纯 LLM 与 LLM+KG（表 10）。

表 10: JD/CV 匹配二分类准确率（剔除 Potential Fit）

方法	准确率 (%)
纯 LLM（结构化 JSON → 预测）	74.3
LLM + KG（JSON + KG 覆盖文本 → 预测）	**85.2**

结论：相对纯 LLM 基线，引入 KG 覆盖分析后准确率提升约 **10.9 个百分点**，证明 GraphRAG 在”语义 + 结构”双重对齐上的有效性。

3.4 简历过度包装检测（SFT + DPO）

3.4.1 任务定义

本任务关注简历中的**语义夸大**（层级与事实不符），而非”AI 生成检测”或文风分类；输出需要可复核——不仅给出结论，还要列出疑点与建议的面试追问，让 HR 可以回落到原文核对。

3.4.2 数据合成

数据与图谱评测同源，清洗对齐后通过 `make_rl_dataset.py` 管道生成 JSONL（`prompt` / `chosen` / `rejected`）；仅保留 Good Fit 与 Potential Fit，排除 No Fit，避免对原本就不匹配的简历”强行拔高”。划分采用固定随机种子，**训练集 1,000 条、测试集 200 条**（约 83% / 17%）。三种合成策略与 DPO 字段的语义见表 11。

表 11: DPO 数据合成策略与字段语义

策略	适用标签	摘要
inflate_weak	Potential Fit	高级词汇堆砌、职责夸大，故意留资历或技术矛盾
inflate_strong	部分 Good Fit	指标夸大 5–20 倍、独揽团队成果等
authentic	部分 Good Fit	真实 CV；chosen 客观、rejected 模拟无理挑刺
字段	**含义**	
prompt	含 JD 与（策略处理后的）CV 的研判指令	
chosen	应提高概率：正确识别夸大或公正评价真实简历	
rejected	应压低概率：误判或挑刺	

3.4.3 训练顺序与评测协议

基座模型选用 **Qwen3.5 4B**，兼顾算力与推理效率。训练顺序为 **SFT → DPO**：SFT 让模型先学会固定的分析范式与输出结构，DPO 在此基础上做成对偏好对齐，抬高可信分析、压低误判。评测时主指标为 **Pairwise Accuracy**，并辅以传统的 Accuracy 与宏平均 F1（与 `is_fabricated` 元数据对齐），基线为同规模的 **Qwen API Zero-shot**。

表 12: 过度包装检测结果（测试集 n = 200）

方法	Accuracy (%)	F1 (%)	Pairwise Acc (%)
直接调用 Qwen API（Zero-shot）	70.5	67.8	73.5
本项目：SFT + DPO	**83.0**	**80.5**	**88.5**

结论：在三项指标上均显著优于 Zero-shot 基线，其中 Pairwise Accuracy 提升 **15 个百分点**，验证了"SFT 定型 + DPO 对齐" 两阶段策略对小模型的适用性。

3.5 微调模型服务化部署

考虑到主站云主机内存受限，过度包装检测模型不适合与业务后端共置。我们将 SFT+DPO 后的 Qwen3.5 4B 权重部署到 **AutoDL** GPU 实例（NVIDIA RTX 5090，Ubuntu 22.04），以独立 **FastAPI** 进程加载并对外暴露 HTTP 接口；阿里云 ECS 与

AutoDL 之间通过 **SSH 端口转发**打通，主业务后端访问 `127.0.0.1:9000/predict` 即可调用远端推理服务。具体的拓扑细节将在第 4 章统一展开。

4 技术栈与部署

4.1 总体架构

ScoutAgent 整体由**前端 SPA**、**后端 API/Agent 服务**、**三类数据存储**组成，统一通过 Docker Compose 编排部署。整体拓扑如图 12 所示。

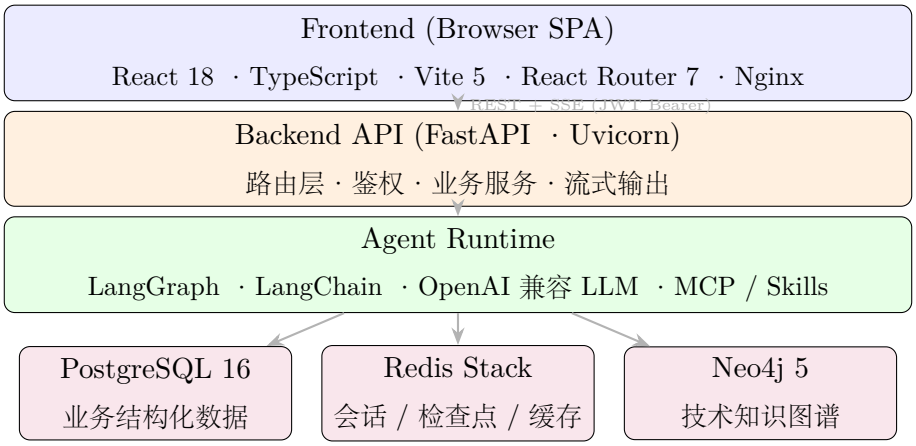

图 12: ScoutAgent 总体技术架构

4.2 前端

前端是一个标准的 React 18 + TypeScript SPA，所有业务路由都受 `ProtectedRoute` 保护，401 自动登出回到登录页。长任务（评估、对话）通过 SSE 流式接收，Markdown 内容统一用 `react-markdown` + `remark-gfm` 渲染。生产环境构建产物由 `nginx:alpine` 静态托管并反向代理到后端（表 13）。

表 13: 前端技术栈

维度	技术选型
UI 框架	React 18
语言	TypeScript 5
构建工具	Vite 5（`@vitejs/plugin-react`）
路由	React Router DOM 7
内容渲染	`react-markdown` + `remark-gfm`（评估报告 / 对话内容）
通信	`fetch` + REST，长任务用 SSE 流式接收
静态托管	生产由 `nginx:alpine` 提供静态资源与反向代理

4.3 后端 API

后端基于 FastAPI + Uvicorn 多 worker 部署，数据访问采用 SQLAlchemy ORM + `psycopg[binary]` 驱动，库表迁移由 Alembic 管理。鉴权使用 JWT（PyJWT）+ Bearer Token，简历 PDF 由 PyMuPDF 解析（多模块同名 `pdfplumber` 仅在抽取实验中使用），流式响应通过 `stream_buffer.py` 维护，全链路 Tracing 接入 LangSmith（表 14）。

表 14: 后端 API 技术栈

维度	技术选型
运行时	Python（`python:slim` 容器）
Web 框架	FastAPI
ASGI 服务器	Uvicorn（`--workers 2`）
数据访问	SQLAlchemy ORM + `psycopg[binary]`
迁移	Alembic
配置管理	Pydantic / `pydantic-settings` + `python-dotenv`
鉴权	JWT（PyJWT）+ Bearer Token
文件解析	PyMuPDF；上传走 `python-multipart`
流式响应	SSE（`stream_buffer.py`）
可观测性	LangSmith Tracing

4.4 Agent 框架

Agent 框架是后端的核心，所有智能体能力都通过 LangGraph 编排（表 15）。LLM 接入统一走 OpenAI 兼容接口（`langchain-openai` + `openai` SDK），默认模型为 Kimi K2.5；token 计算用 `tiktoken`；记忆侧的状态/检查点由 LangGraph Checkpoint Redis 持久化；工具扩展同时支持 MCP 协议与自研 Skills 模块；知识图谱检索使用 Neo4j 官方 Python Driver 直连。

表 15: Agent 框架技术栈

维度	技术选型
编排框架	LangGraph（多阶段 Workflow、Planning、状态机）
基础能力	LangChain（Prompt / Tool / Chain 抽象）
LLM 接入	OpenAI 兼容接口（`langchain-openai` + `openai`）
Token 计算	`tiktoken`
状态 / 记忆	LangGraph Checkpoint，由 Redis 持久化
工具扩展	MCP（`langchain-mcp-adapters` + `mcp`）+ 自研 Skills
知识图谱检索	Neo4j 官方 Python Driver
业务模块	`agent/planning`, `assessment`, `chat`, `resume`, `skills`, `tools`

4.5 数据存储

三类存储承担不同职责：PostgreSQL 持久化业务结构化数据、Redis 承担会话与检查点、Neo4j 提供 GraphRAG 检索（表 16）。

表 16: 数据存储

存储	版本 / 镜像	用途
PostgreSQL	`postgres:16` / `postgres:alpine`	企业、JD、候选人、CV、申请、评估等结构化业务数据
Redis	`redis/redis-stack-server:latest`	会话状态、缓存、LangGraph Checkpoint 持久化
Neo4j	`neo4j:5` / `neo4j:latest`	技术知识图谱（技能、框架、上下位关系），支撑 GraphRAG

4.6 部署

系统采用 Docker Compose 双 compose 编排——开发用 `docker-compose.yml`、生产用 `docker-compose.prod.yml`（表 17）。各服务均配置 `healthcheck`（`pg_isready`、`redis-cli ping`、Neo4j HTTP、`/api/health`），生产 compose 进一步对 PostgreSQL 与 Neo4j 做 JVM/缓冲调优并设置 memory limits。

表 17: 部署方案

维度	方案
容器编排	Docker Compose（开发 / 生产双文件）
服务拓扑	frontend (Nginx 80) → backend (FastAPI 8000) → postgres / neo4j / redis
镜像构建	后端基于 `python:slim`；前端 `node:18-alpine` 多阶段构建后由 `nginx:alpine` 托管
健康检查	各服务均配置 healthcheck
资源限制	生产 compose 设置 memory limits/reservations，并对 PG/Neo4j 做调优
配置注入	`.env` + 环境变量（数据库连接、JWT、LLM Key、Neo4j 凭据等）
本地一键启动	`run.sh` 串行启动 Neo4j → PG/Redis → Backend → Frontend

4.7 云端运行环境与 AutoDL 推理隧道

主站部署在阿里云 ECS（Ubuntu 22.04 LTS、2 核 / 4 GB），Nginx、SSL 证书与反向代理通过宝塔面板管理；公网入口通过弹性公网 IP 对外提供。**过度包装检测模型**的推理服务部署在 AutoDL GPU（NVIDIA RTX 5090）上，通过 **SSH 本地端口转发**将 AutoDL 上 `127.0.0.1:8000` 的 FastAPI 映射到 ECS 本机 `127.0.0.1:9000`（图 13）。这种”主业务 + AutoDL 自托管推理”的混合架构有两个好处：(1) 避免在 2 GB 内存的 ECS 上加载 4B 级权重导致 OOM；(2) 推理算力按任务时段租用，不为整站长期绑定高价 GPU。

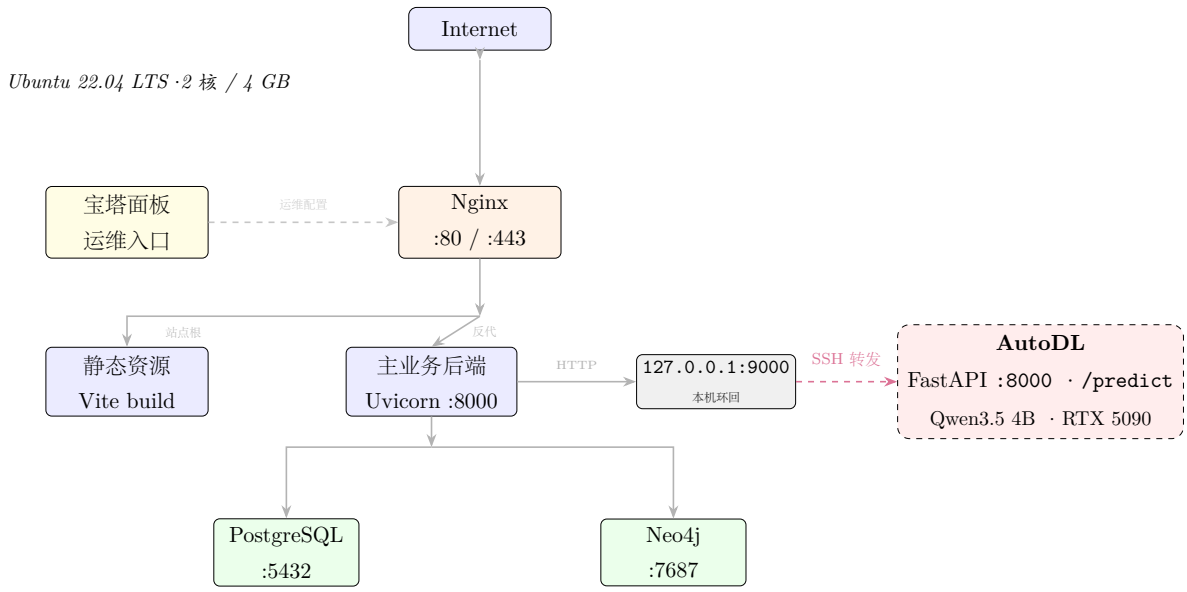

图 13: 部署拓扑：Web、数据层与 AutoDL 推理（SSH 端口转发）

SSH 隧道的典型命令为 `ssh -L 9000:127.0.0.1:8000 root@<AutoDL 实例地址>`；生产环境可使用 `autossh` 或在 `systemd` 中注册服务以保持自动重连。其余智能体（如 JD-CV 匹配、深度对话）继续通过 OpenAI 兼容接口调用云端 Kimi 模型，与上述自托管 Qwen 推理路径在 `.env` 中分别配置 Base URL 即可并行存在。

5 系统功能展示

5.1 页面总览

ScoutAgent 前端基于 React 18 + TypeScript + Vite + React Router 7 构建，由左侧固定的图标导航栏串联起 5 个核心页面，所有页面共享 JWT 鉴权、SSE 流式通信与统一的 Markdown 渲染体系（表 18）。

表 18: ScoutAgent 五大业务页面

#	页面	路由	一句话定位
1	企业与岗位管理	`/enterprises`	招聘信息的”源头”：维护客户、JD 与投递关系
2	候选人管理	`/candidates`	简历的”档案库”：PDF → 结构化 Profile
3	批量化评估	`/assess`	全局视角的”流水线评估”：批量打分与筛选
4	**深度评估 Agent**	`/assess/assistant`	**对话式深评：多轮、可追溯、有记忆**
5	Eval 系统	`/eval`	离线评测中心：跑评测、看指标、做回归

五个页面在业务上构成闭环：第 1、2 页提供”原料”（企业/JD 与候选人简历），第 3 页做”粗筛”，第 4 页做”精评 + 经验沉淀”，第 5 页保证”模型升级不退化”（图 14）。

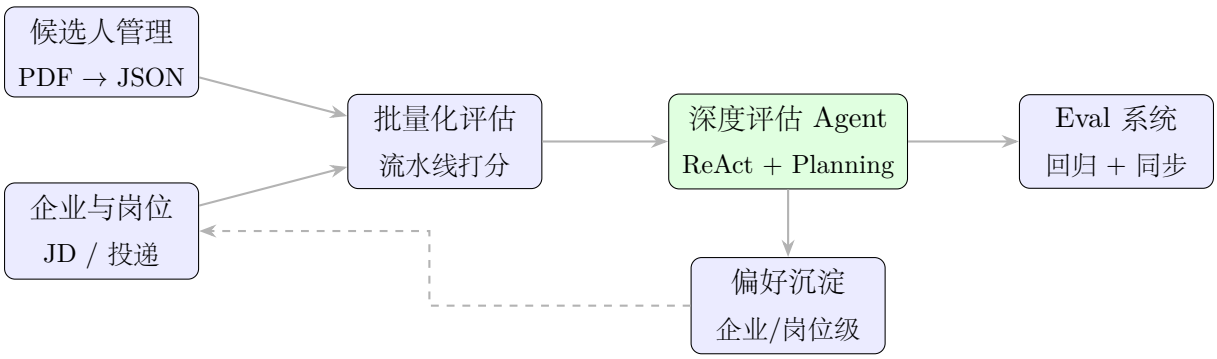

图 14: 五大页面的业务闭环

5.2 企业与岗位管理

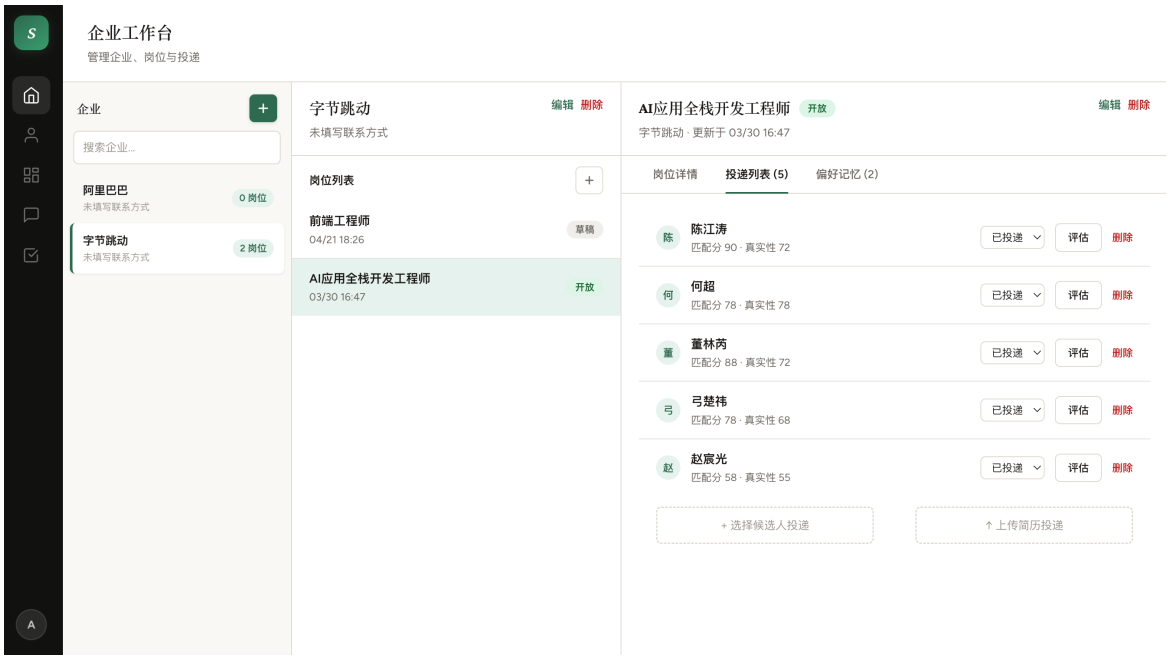

图 15: 企业与岗位工作台：三栏布局

企业与岗位管理页是**招聘流程的入口**，所有评估的上游数据都在此创建与维护。页面采用三栏式工作台：

- **左栏 · 企业列表**：企业的搜索、新建、编辑、删除；
- **中栏 · 岗位列表**：进入企业后展示其下所有 JD，支持新建岗位与状态切换（草稿 / 开放 / 关闭）；
- **右栏 · 岗位详情**：以 Tab 形式分别展示岗位详情、投递列表与**偏好记忆**（由 Agent 自动沉淀的企业级与岗位级偏好，支持人工删除）。

这一页是后续所有评估的”上下文池”——后端记忆系统会持续向这里 upsert 偏好条目。

5.3 候选人管理

图 16: 候选人库：简历的全生命周期管理

候选人管理页负责把非结构化 PDF 简历转化为可检索、可复用的候选人 Profile。左栏为候选人列表（按姓名/邮箱搜索），右栏以多 Tab 展示候选人详情：原始 PDF（浏览器内嵌预览）、解析简历（结构化字段）、原文（纯文本）、JSON（完整 `parsed_json`）、投递记录。上传 PDF 后自动启动 `parse_status` 轮询（pending → parsing → completed / failed），无需手动刷新。该页的解析能力对应第 3 章的 Qwen + CoT 模块，F1 由 66% 提升到 78%。

5.4 批量化评估看板

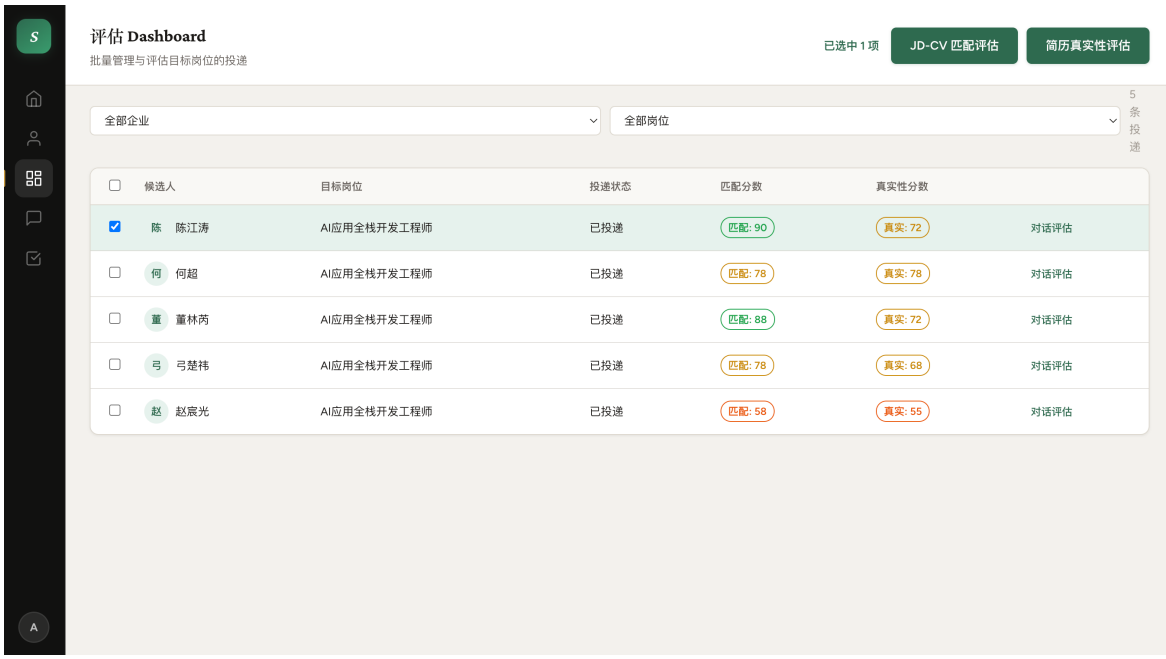

图 17: 批量化评估看板：双任务并列触发

批量化评估页是”流水线作业”形态：用户先按”企业 / JD”两级下拉过滤投递列表，再批量勾选目标投递；顶部并列两个批量任务按钮——**JD-CV 匹配评估**（5 维度：技能匹配、经验相关性、教育背景、行业契合度、综合潜力）与**简历真实性评估**（5 维度：时间线连贯性、职责合理性、成就可信度、信息完整性、一致性）。每行结果以双分数徽章呈现（匹配分 + 真实性分），按 80/60/40 分阈值自动染色（绿/黄/橙/红）；点击分数即可弹出评估报告 Modal，包含环形分数、等级徽章、维度条形图与优势/不足/风险点列表。这一页的目标是”快”——给出量化结论让 HR 先压缩漏斗，深度判断交给第 5.5 节。

5.5 深度评估 Agent（重点）

图 18: 深度评估 Agent：三栏对话界面与 Plan 可视化

深度评估 Agent 是项目的**核心交付物**：把”打分流水线”升级为”会调工具、有记忆、能规划”的 Agent，让 HR 能像和资深面试官对话一样深挖每一份候选人或每一个岗位。

5.5.1 整体布局与两种对话模式

页面顶部为模式切换条，主体为三栏布局——左栏为投递/岗位列表（含候选人勾选区），中栏为多轮对话区（含会话 Tab 与消息流），右栏为上下文与能力面板（Context Tab / Capabilities Tab）。两种对话模式的差异见表 19。

表 19: 深度评估 Agent 的两种对话模式

模式	触发条件	上下文范围	适用场景
`application` 投递对话	选中一条投递记录	单候选人 × 单 JD	单点深挖：一个人到底合不合适
`jd` 岗位对话	选中 JD + 勾选若干候选人	一个 JD × N 个候选人	横向比较：在 N 个候选人里选谁

5.5.2 会话管理：持久化与断流恢复

这是该页工程含量最高的部分。系统为每条投递 / 每个 JD 都维护独立的历史会话列表（`listChatSessions`），支持多 Tab 切换、新建、删除。跨页面持久化通过 `sessionStorage` 同时缓存 `assess_chat_nav`（当前模式 / 选中 ID / `sessionId` / `runId` / `lastSeq`）和 `assess_chat_msgs`（完整消息列表），即使用户跳到别的页面再回来，对话状态、Plan 步骤、已收 token 都能原样恢复。如果用户在 Agent 还在流式输出时离开页面，回来后会自动调用 `resumeAgentStream(runId, lastSeq)` 从断点续传，恢复失败则 fallback 到 `listChatMessages` 重新拉取历史。

5.5.3 SSE 七类流式事件渲染

后端通过 SSE 推送七类事件，前端用 `chatWithAgentStream` 的 `ChatStreamHandlers` 分别接管（表 20）。

表 20: SSE 事件类型与前端反应

事件	前端反应
`onSession`	拿到 `session_id` / `run_id`，刷新会话 Tab
`onEventSeq`	维护 `lastSeqRef`，用于断流续传定位
`onThinking`	渲染顶部”正在制定执行计划…/ 执行步骤 3/7 …”进度条
`onPlan`	把 plan 数组渲染为 **PlanTracker** 组件
`onStepDone`	把对应步骤标记为完成，下一步置为 executing
`onReplan`	保留已 done 的步骤，后续步骤替换为新计划
`onToken`	把 token 增量 append 到 Markdown 气泡
`onDone` / `onError`	收尾、停止 spinner、错误兜底

5.5.4 Planning 模式可视化

输入框右侧的 Plan 切换按钮控制后端 `use_planning` 字段。开启后，用户消息送出后立即出现一条 **PlanTracker** 气泡（占位、空步骤）；收到 `onPlan` 后渲染步骤列表，第一步置为 executing（带 spinner），其余为 pending（灰点）；每收到 `onStepDone` 即将该步打勾并展示一段 `result_preview`，下一步自动接力；`onReplan` 则保留已完成步骤，后续重排——这正是第 2 章中 Plan-and-Execute 范式的前端可视化。最终 Markdown 报告以新气泡追加在 PlanTracker 之后。

5.5.5 右栏：上下文与能力面板

右栏的 **Context Tab** 根据当前模式动态展示：投递模式下提供候选人卡 + 简历摘要 + 岗位信息 + 投递状态 + 历史评估报告 + 企业/岗位偏好；岗位模式下展示岗位信息卡 + ”已加入讨论”候选人列表 + 偏好。**Capabilities Tab** 通过 `getAgentCapabilities` 拉取后端实际注册的能力清单，分技能模块（Skills）、内置工具（Tools）与 MCP 服务三类透明展示，让 HR 知道 AI 此刻能调哪些外部能力，不再是黑盒。

5.5.6 与 Agent 三要素的呼应

把 5.5.1–5.5.5 串起来看，正好对应第 2 章给出的”为什么必须是 Agent”的三要素（表 21）。

表 21: 深度评估 Agent 页面对 Function Calling / Memory / Planning 的可见体现

Agent 要素	**在该页的具体体现**
Function Calling	Capabilities Tab 中的 Tools / Skills / MCP；Context 里的简历、JD、KG 偏好都是工具产物
Memory	会话级 Tab 条 + sessionStorage + 断流恢复；Context Tab 中的”企业 / 岗位偏好”是长期记忆
Planning	Plan 切换按钮 + PlanTracker + Replan 事件，把 LangGraph 的多步规划与重规划完整可视化

这也是该页与第 5.4 节”批量评估”最本质的区别——**批量评估只能给一个分数，深度评估 Agent 能讲清楚”为什么是这个分数”，并支持继续追问。**

5.6 Eval 系统

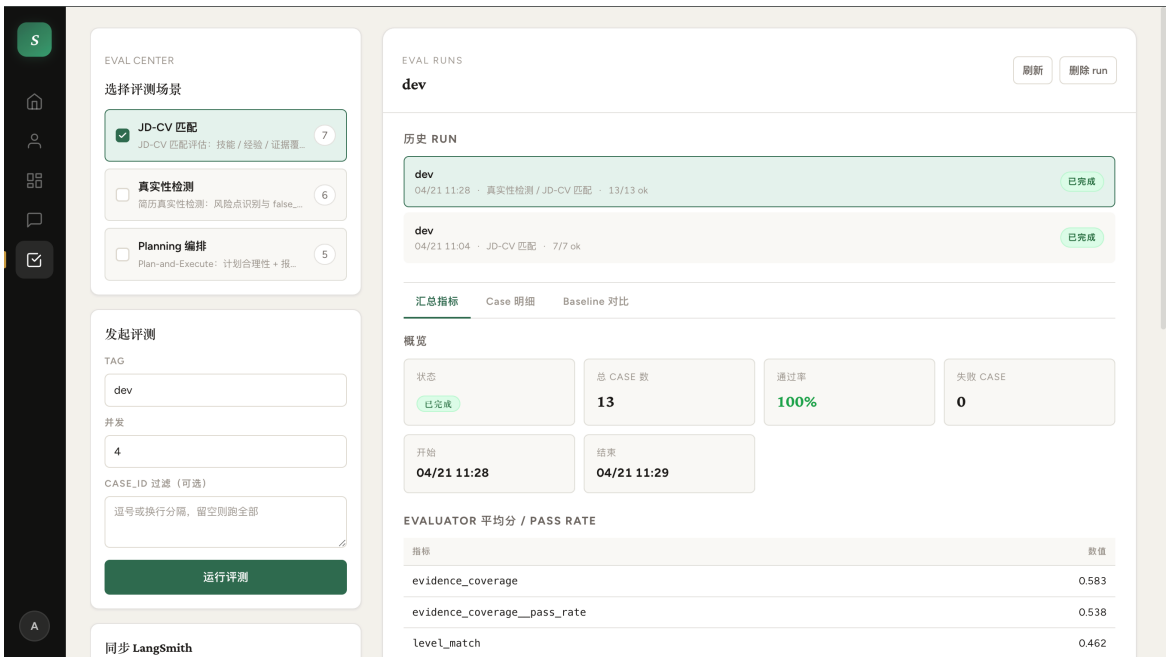

图 19: Eval 系统：离线评测中心

Eval 系统是面向开发者的离线评测工作台，不在业务流上，专门用于回归 Agent 质量。三大评测场景与后端 `evaluators` 对齐：`jd_cv_match`、`authenticity`、`planning`。主要功能区包括：

- **创建 Run**：勾选场景、设置 tag（如 `dev` / `prod`）、并发数与 case 过滤器，一键 `createEvalRun`；
- **Run 列表**：所有历史 run 状态徽章（排队 / 运行 / 完成 / 失败），运行中的 run 自动 4 秒轮询刷新进度；
- **详情面板（三 Tab）**：Summary 展示 `aggregate_scores`（通用指标）与 `business_metrics`（场景业务指标）；Cases 列出每条 case 的输入 / 期望 / 实际输出 / evaluator 子分；Compare 在选定 baseline run 后做 `compareEvalRuns`，逐指标显示 Δ 与方向（区分”越大越好”/”越小越好”）；
- **数据集同步**：`syncEvalDataset` 一键把当前评测集同步到 LangSmith dataset。

这一页让”每一次模型 / Prompt / Skill 的改动”都能被定量回归，是 Agent 项目能持续迭代的工程基石。

6 结论与展望

6.1 核心结论

围绕招聘行业”关键词匹配失效、简历过度包装、能力评估粗糙、信息碎片化”四类核心痛点，本项目以 ScoutAgent 为载体给出了一套从需求到落地的完整方案。三项关键算法在与基线的对照实验中均取得显著提升：

- **结构化信息抽取**：F1 由 66%（Zero-shot）提升至 78%（Qwen + CoT），简历评测样本量 5,960；
- **GraphRAG 匹配增强**：JD/CV 二分类准确率由 74.3%（纯 LLM）提升至 85.2%（LLM + KG），相对提升约 10.9 个百分点；图谱多跳推荐在 Hit@5 与 MRR 上同步优于共现与词频基线；
- **过度包装检测**：Pairwise Accuracy 由 73.5%（Zero-shot）提升至 88.5%（SFT + DPO），F1 同步从 67.8% 提升至 80.5%。

工程上，本项目以 LangGraph 为编排核心同时落地了 ReAct 与 Plan-and-Execute 双范式，统一组合 Tools / Skills / MCP 三层工具，并通过 Redis Checkpointer 与 PostgreSQL 偏好库实现两层记忆体系。前端则把 Function Calling、Memory、Planning 三要素以 Capabilities Tab、会话恢复与 PlanTracker 三种形态可视化呈现给 HR，使 Agent 不再是黑盒。

6.2 局限与未来工作

当前系统仍存在若干局限，对应几个明确的后续方向：

1. **DPO 偏好闭环**：当前 DPO 数据由半自动管道合成，未来可以从 HR 实际反馈中持续采集偏好对，形成线上闭环；
2. **岗位维度综合决策 Agent**：当前评估仍以单投递为粒度，未来可在岗位池上做相对排序与团队互补性分析；
3. **跨企业偏好迁移**：当前偏好在企业级与岗位级两层 scope 内沉淀，未来可探索跨行业、跨岗位的偏好迁移机制；
4. **多智能体代码评测**：未来可在 Sandbox 环境中引入代码执行评测 Agent，对候选人提交的代码片段做运行验证与过程评分，进一步增强能力评估的可量化程度。

———报告完———